DuoWaves: MPI+OpenMP and CUDA Implementation Analysis

Martinelli Luca (2129604), Salvati Antonio (2050590)

14 October, 2026

1 Computational Structure and Correctness

1.1 Algorithmic Structure and Work Model

The core simulation models the cumulative energy deposition across a 1D spatial layer of size L over a discrete sequence of S storms. Each storm s_i carries P_i particles defined by an impact coordinate $x_j \in [0, L-1]$ and an initial energy E_j. The algorithm executes three strictly ordered phases per storm:

1. **Bombardment:** Particle impacts are distributed across all cells $k \in [0, L-1]$. The injected energy attenuates with distance $d = |x_j - k| + 1$ via $E_k = E_j/(L \cdot \sqrt{d})$. An energy contribution is accumulated into the layer if and only if $|E_k| \geq \tau/L$, where τ is the global threshold parameter.
2. **Relaxation:** A 3-point moving average models thermal smoothing across the layer:
$$L_k^{(t+1)} = \frac{L_{k-1}^{(t)} + L_k^{(t)} + L_{k+1}^{(t)}}{3} \quad \forall k \in [1, L-2] \quad (1)$$
3. **Peak Selection:** The layer is scanned to find the highest value among interior cells that are strictly greater than both their left and right neighbors.

Temporal dependencies require storm s_{i+1} to process the fully relaxed state of storm s_i. The sequential baseline exhibits an asymptotic complexity of:

$$W_{\text{seq}} = \mathcal{O}\left(\sum_{i=1}^{S} (P_i \cdot L + L)\right) \approx \mathcal{O}(S \cdot \bar{P} \cdot L) \quad (2)$$

The computational load is heavily compute-bound within the bombardment phase due to repeated calls to `sqrtf()`, and memory-bound during relaxation due to sequential read/write passes.

1.2 Boundary Treatment and Tie-Breaking Policy

Boundary cells ($k = 0$ and $k = L-1$) participate in energy reception during bombardment but are explicitly excluded from smoothing in the relaxation phase, preserving their pre-relaxation values.

During peak selection, candidates are evaluated exclusively over interior cells ($k \in [1, L-2]$). A cell qualifies as a local peak if:

$$L_k > L_{k-1} \quad \wedge \quad L_k > L_{k+1} \quad (3)$$

A valid peak becomes the storm's new maximum if its energy is strictly greater than the current maximum ($L_k > M_i$). Because the code updates only when a value is strictly greater, if two cells have the exact same maximum energy, the one with the smaller index k is kept. If no cell qualifies as a peak during the storm, the default values ($M_i = 0.0\text{f}, \text{pos}_i = 0$) stay the same.

1.3 Numerical Correctness Verification

Parallel execution reorders floating-point additions, inducing non-associative round-off variations. Correctness is verified against the serial driver: recorded positions must match identically ($\text{pos}_{\text{par}} = \text{pos}_{\text{ref}}$), while maximum energy levels are validated within a strict relative tolerance $|M_{\text{par}} - M_{\text{ref}}|/M_{\text{ref}} < \varepsilon$.

2 MPI+OpenMP Implementation

The hybrid approach combines distributed message passing across nodes using MPI[1] with shared-memory multi-threading inside each node via OpenMP[2]. This section describes the architecture of the solution, documents the iterative trial-and-error optimization across development branches, and presents the performance evaluation on the Sapienza's cluster.

2.1 Domain Decomposition and Workload Distribution

The 1D spatial domain of size L is partitioned into contiguous blocks across MPI processes using a standard 1D domain decomposition:

$$\text{local_chunk} = \lfloor L/\text{size} \rfloor, \quad \text{resto} = L \pmod{\text{size}} \quad (4)$$

Ranks with rank $<$ resto receive one additional cell (local_size = local_chunk + 1), distributing the domain evenly without omitting any cells.

Each process allocates local_size + 2 elements to hold two ghost cells: index 0 for the left neighbor and index local_size + 1 for the right neighbor. The buffer allocation employs `posix_memalign` with 64-byte alignment. This matches the cache-line boundary on x86-64 CPUs, avoiding cache line splits.

2.2 Trial and Error: Iterative Optimization Pipeline

The optimization path progressed through four "main" successive branches to locate and eliminate architectural

bottlenecks:

2.2.1 Branch 1: Sqrt_Optimization (First working-branch)

This initial version introduced 1D domain decomposition, ghost cells, and an $\mathcal{O}(1)$ pointer swap between layers instead of sequential buffer copying:

```
float *temp = layer_copy;
layer_copy = layer;
layer = temp;
```

However, three major performance bottlenecks remained:

- **Precomputed square root table:** Precomputing square roots into an allocated array (`sqrt_table`) produced continuous cache misses across L1/L2 caches. Accessing main memory proved significantly slower than executing hardware `sqrtf` instructions directly on CPU floating-point units (it can take arround 100 to 200 clock cycles).
- **Atomic contention:** OpenMP parallelization was assigned to the outer particle loop (j), requiring atomic operations to avoid concurrent write corruption:

```
#pragma omp parallel for private(j, k)
for (j = 0; j < storms[i].size; j++) {
    for (k = 1; k <= local_size; k++) {
        if (energy_k >= THRESHOLD / layer_size ||
            energy_k <= -THRESHOLD / layer_size) {
            #pragma omp atomic
            layer[k] += energy_k;
        }
    }
}
```

  This induced heavy bus contention and continuous cache-line bouncing among active cores.
- **Lock serialization:** The local peak search was guarded by `omp_lock_t`, forcing threads to serialize at every candidate peak:

```
omp_set_lock(&writelock);
if (layer[k] > local_maximum) {
    local_maximum = layer[k];
    local_positions = offset + k - 1;
}
omp_unset_lock(&writelock);
```

2.2.2 Branch 2: Layers_Optimization (Literally a Roll-Back)

The unused square-root lookup table was eliminated to reclaim memory bandwidth. The core computation was refactored into a modular helper function:

```
static void update(float *layer, int layer_size, int k,
                   int pos, float energy, int global_k);
```

Marking the helper as `static` allowed the compiler (`-O3`) to perform automatic inlining while keeping the source modular. However, the slowdowns caused by atomic operations during bombardment and locks during the peak search were still present.

2.2.3 Branch 3: AddParallelZone (Key Architectural Shift)

This version introduced a persistent parallel region across all storms and restructured the computation to remove both atomic operations and locks:

- **Loop inversion and register accumulation:** Swapping loop nesting placed the parallel loop over layer coordinates (k). Because each thread is granted exclusive ownership over a slice of cells, atomic operations were eliminated entirely:

```
#pragma omp for
for (k = 1; k <= local_size; k++) {
    float save_cell_energy = layer[k];
    for (j = 0; j < storms[i].size; j++) {
        if (fabsf(energy_k) >= local_threshold)
            save_cell_energy += energy_k;
    }
    layer[k] = save_cell_energy;
}
```

  Accumulating values into the scalar variable `save_cell_energy` keeps intermediate sums inside CPU registers, flushing to RAM only once per cell.
- **Lock-free per-thread peak reduction:** Locks were replaced with a private array indexed by thread ID, scanned by the master thread prior to global reduction:

```
#pragma omp for
for (k = start; k <= end; k++) {
    if (layer[k] > layer[k-1] && layer[k] > layer[k+1]) {
        if (layer[k] > my_data.local_maximum) {
            my_data.local_maximum = layer[k];
            my_data.local_positions = offset + k - 1;
        }
    }
}
threads_results[my_id] = my_data;
```

2.2.4 Branch 4: main (Non-Blocking Network Communication)

In the final submitted version, sequential blocking `MPI_Sendrecv` exchanges were upgraded to four simultaneous non-blocking operations:

```
MPI_Request req[4];
MPI_Isend(&layer[local_size], 1, MPI_FLOAT, right, 0, MPI_COMM_WORLD, &req[0]);
MPI_Irecv(&layer[0],          1, MPI_FLOAT, left,  0, MPI_COMM_WORLD, &req[1]);
MPI_Isend(&layer[1],          1, MPI_FLOAT, left,  0, MPI_COMM_WORLD, &req[2]);
MPI_Irecv(&layer[local_size+1], 1, MPI_FLOAT, right, 0, MPI_COMM_WORLD, &req[3]);
MPI_Waitall(4, req, MPI_STATUSES_IGNORE);
```

Sending and receiving all messages at the same time allows data to travel in both directions at once, so the total communication time is only limited by the slowest transfer.

2.3 Performance and Scaling Evaluation

The performance of the final submitted branch was benchmarked on the cluster by scaling execution from 1 up to 128 total cores. Each data point represents the median of 5 independent runs to account for system variance.

2.3.1 Strong Scaling Speedup and Efficiency

Figure 1 and Figure 2 display the strong scaling results. At a single core ($p = 1$), the code exhibits an apparent superlinear speedup between $5\times$ and $9\times$ compared to

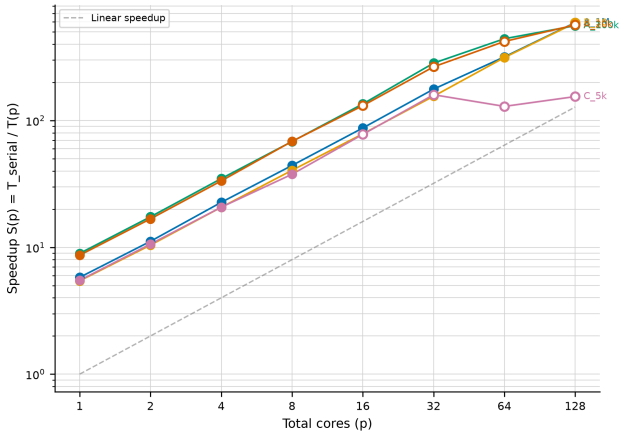


Figure 1: *Hybrid strong scaling speedup across heterogeneous workload families relative to the sequential baseline.*

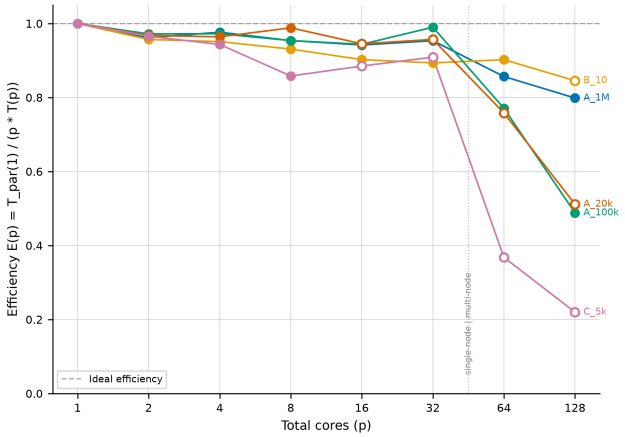


Figure 2: *Parallel efficiency in strong scaling highlighting the single-node to multi-node transition at $p = 32$.*

the sequential reference. This speedup is due to local optimizations present in the hybrid code: register accumulation, 64-byte memory alignment via `posix_memalign`, and $O(1)$ pointer swapping.

For heavy workloads (`A_1M` and `A_100k`), the speedup scales almost linearly up to 128 cores, reaching nearly 600× acceleration. In this case, the time spent calculating particle impacts is much larger than the time spent exchanging data over the network.

In Figure 2, within a single node ($p \leq 32$), parallel efficiency remains exceptionally high, between 90% and 100% across all workloads, because threads and processes communicate directly through shared memory. Beyond 32 cores, execution crosses into multi-node territory and messages must travel through the network. While large workloads retain an efficiency of 80%–85% at 128 cores, the smallest dataset (`C_5k`) drops sharply around 20%. For small inputs, local computation becomes too brief relative to the fixed latency of `MPI_Waitall` and `MPI_Allreduce`, following Amdahl's Law[3].

2.3.2 Hybrid Weak Scaling

Figure 3 presents the weak scaling evaluation, where the workload per core is kept constant from $p_0 = 4$ up

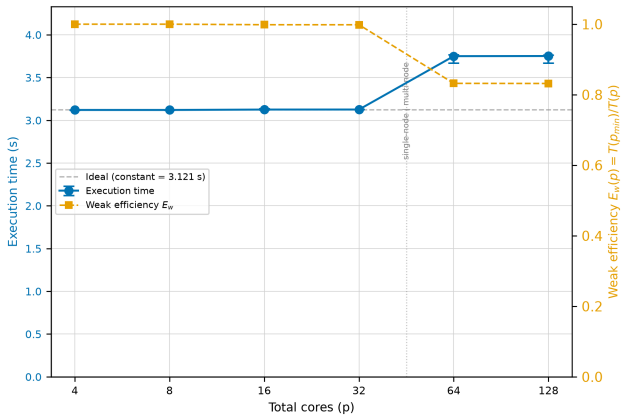


Figure 3: *Hybrid weak scaling runtime and weak efficiency relative to $p_0 = 4$, showing multi-node network transition.*

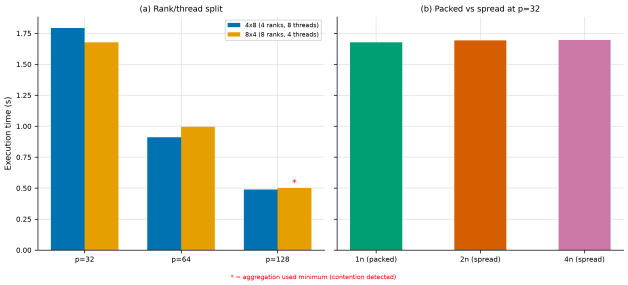


Figure 4: *Configuration study: (a) MPI rank vs OpenMP thread split ($p \in [32, 128]$); (b) packed vs spread allocation at $p = 32$.*

to $p = 128$. Within the single-node domain ($4 \leq p \leq 32$), the execution time remains flat around at 3.1 s, corresponding to an ideal weak efficiency of 100%.

Moving to multiple nodes ($p \geq 64$), the runtime increases by approximately +0.63 s to 3.75 s (83% efficiency). This step represents the added latency of network interface cards and network switches. Importantly, runtime stays flat between 64 and 128 cores, confirming that point-to-point halo exchanges scale with $\mathcal{O}(1)$ network complexity.

2.3.3 Process/Thread Mapping and Affinity

Figure 4 explores how processes and threads are placed on the cluster hardware.

In Figure 4(a), comparing 4×8 (4 MPI ranks, 8 OpenMP threads each) with 8×4 (8 MPI ranks, 4 threads each) reveals a clear trade-off:

- At $p = 32$ (single node), 8×4 is slightly faster (1.68 s vs 1.79 s) because intra-node MPI incurs no network latency, and smaller OpenMP loops reduce scheduling overhead.
- At $p = 64$ and $p = 128$ (multi-node), 4×8 achieves better performance (0.91 s vs 1.00 s at $p = 64$). Halving the number of MPI ranks per node cuts the number of active network endpoints in half, lowering network contention during reductions.

Figure 4(b) tests $p = 32$ cores allocated in three ways: 1 full node (*packed*), 2 nodes with 16 cores each (*spread*), and 4 nodes with 8 cores each (*spread*). Runtime stays constant at ≈ 1.69 s across all layouts. This proves that

memory bandwidth is not saturated; loop inversion and register accumulation kept the code compute-bound, making execution insensitive to distributed node placement at this scale.

3 CUDA Implementation

The CUDA[4] GPU implementation executes bombardment, relaxation, and peak selection through three kernels: `bomb`, `relax`, and `reduce_argmax`. Storms remain sequential, while cells within each storm are processed in parallel.

3.1 Memory Organization and Workload Distribution

The pointer-based storm array is flattened into a contiguous array of position–energy pairs and transferred to device memory. An `offsets` array marks each storm's start and includes a final entry marking the end of the data:

```
unsigned int base = d_offsets[i_storm];
int num_particles =
    (d_offsets[i_storm + 1] - base) / 2;
```

Two device buffers hold the layer before and after relaxation. Bombardment and relaxation use one-dimensional grids with one thread per cell; bounds checks handle the partially filled final block.

3.2 Trial and Error: Iterative Optimization Pipeline

3.2.1 Branch 1: cuda-first (Atomic Updates)

The initial two-dimensional grid assigned one thread to each cell–particle pair. Contributions to the same cell were combined using `atomicAdd`:

```
// Cell-particle mapping: multiple writers per cell
if (fabsf(energy_k) >= calc_threshold)
    atomicAdd(&d_layer[i_layer], energy_k);
```

Atomics prevented lost updates but introduced contention. The addition order also depended on thread execution, allowing floating-point rounding differences between runs.

3.2.2 Branch 2: cuda-1D (Exclusive Cell Ownership)

As in the OpenMP loop inversion, each thread was assigned a single cell and processed particles in input order. Exclusive ownership removed atomic updates and stabilized the accumulation order. This branch also preallocated reduction buffers instead of allocating and freeing them at every pass.

3.2.3 Branch 3: cuda-shared (Particle Tiling)

Particles were loaded cooperatively into shared memory in tiles of at most `BLOCK_SIZE` elements. Each block reused the loaded positions and normalized energies across its cell threads:

```
// Inside the particle-tile loop
__syncthreads(); // Previous tile is no longer in use
if (threadIdx.x < tile_count) {
    int p = base + 2 * (tile + threadIdx.x);
    s_pos[threadIdx.x] = d_flattened[p];
    s_energy[threadIdx.x] =
        ((float)d_flattened[p + 1] * 1000)
        / layer_size;
}
__syncthreads(); // Tile is ready for all threads
```

Threads outside the layer domain must still reach both barriers [4]. Only the cell computation is guarded by the bounds check. This branch still allocated reduction buffers at each pass.

3.2.4 Branch 4: main (Local Accumulation and Buffer Reuse)

The final version combined shared-memory tiling with a local accumulator, writing each cell to global memory only once:

```
float save_cell_energy = 0.0f;
if (active) save_cell_energy = d_layer[i_layer];
// For each tile, after cooperative loading:
if (active) {
    for (int j = 0; j < tile_count; j++) {
        int distance = abs(s_pos[j] - (int)i_layer) +
      1;
        float energy_k = s_energy[j]
                         / sqrtf((float)distance);
        if (fabsf(energy_k) >= calc_threshold)
            save_cell_energy += energy_k;
    }
}
// After all tiles:
if (active) d_layer[i_layer] = save_cell_energy;
```

Two preallocated pairs of value/index arrays are alternated between reduction passes (*ping-pong*). Each holds as many candidates as the first pass produces, which also covers all later passes. Separate input and output buffers prevent interference between blocks; allocation occurs once before the storm loop.

3.3 Relaxation and Maximum Reduction

After bombardment, the device layer pointers are swapped, as in the hybrid implementation. The `relax` kernel reads the old buffer and writes the new one, preserving boundary values:

```
if (i == 0 || i == layer_size - 1)
    layer[i] = layer_copy[i];
else if (i < layer_size - 1)
    layer[i] = (layer_copy[i-1] + layer_copy[i]
                + layer_copy[i+1]) / 3.0f;
```

The first `reduce_argmax` pass selects strict interior local peaks; other threads contribute `(-INFINITY, -1)`. Each block performs a shared-memory tree reduction, halving the comparison stride at every step. Equal values retain the lowest valid cell index:

```
bool better = sdata[tid + s] > sdata[tid];
bool tie_lower = sdata[tid + s] == sdata[tid]
    && sidx[tid + s] >= 0
    && (sidx[tid] < 0 || sidx[tid + s] < sidx[tid]);
if (better || tie_lower) {
    sdata[tid] = sdata[tid + s];
    sidx[tid] = sidx[tid + s];
}
```

Thread 0 writes the block's winner. Further kernel passes reduce these winners without repeating the local-peak test, until one candidate remains. The host updates the storm result only for a valid index and a value

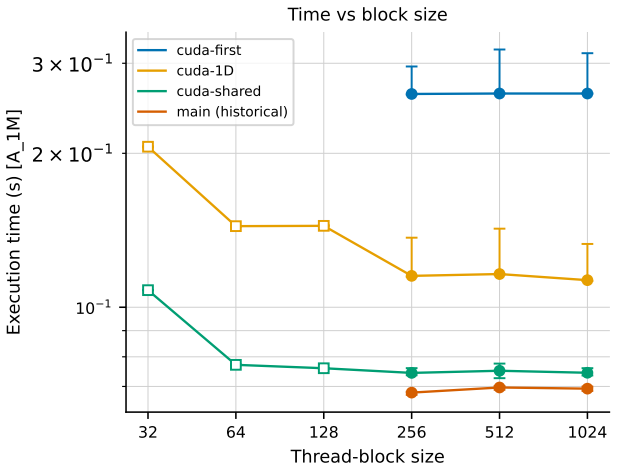


Figure 5: *CUDA runtime on* `A_1M` *versus block size: medians and min–max ranges over five valid runs. Open squares mark corrected* `node122` *reruns; circles mark historical* `node111` *measurements. Historical* `main` *predates the final grid fix.*

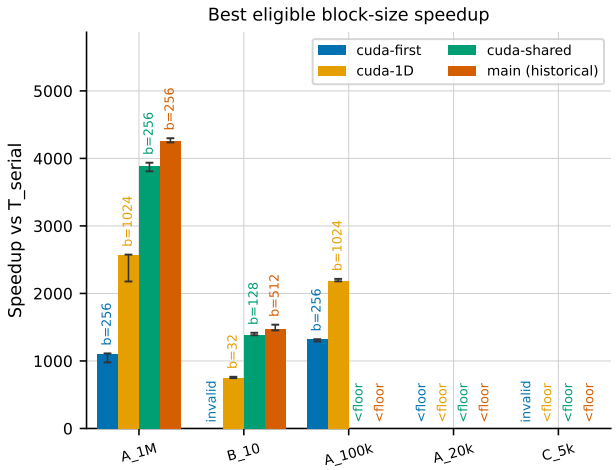


Figure 6: *CUDA speedup using the lowest eligible median for each branch and workload. Labels show the selected block size; error bars bound the observed timing ratios. The label invalid denotes no fully valid configuration; <floor denotes no eligible median at or above* 0.02 *s. Historical* `main` *predates the final grid fix.*

above the initial maximum, preserving the default $(0, 0)$ when no positive peak qualifies.

3.4 Launch Configuration and Correctness Fixes

In `cuda-1D` and `cuda-shared`, the relaxation grid originally assumed 256 threads while the launch used `BLOCK_SIZE`. Smaller blocks therefore left cells unprocessed. The formula was corrected; `main` uses the same grid for both kernels:

```
int grid = (layer_size + BLOCK_SIZE - 1) / BLOCK_SIZE
    ;
bomb<<<grid, BLOCK_SIZE>>>(/* arguments omitted */);
relax<<<grid, BLOCK_SIZE>>>(d_layer, d_layer_copy,
                            layer_size);
```

Every block-size configuration must cover the full layer and match the sequential reference in peak positions and values. Timings from incomplete layer coverage are excluded from valid performance comparisons.

3.5 Performance Evaluation

CUDA results use one GPU, with five repetitions per configuration; plots report medians and min–max ranges. The timed interval includes the complete `core()` call (allocation, transfers, kernels and cleanup) and final synchronization, excluding input reading and initial synchronization. Speedup is $S_{\text{CUDA}} = T_{\text{seq}}/T_{\text{CUDA}}$. Varying threads per block measures launch tuning, not multi-GPU strong or weak scaling.

3.5.1 Branch Comparison

At 256 threads per block, `A_1M` runtime falls from 0.2613 s (`cuda-first`) to 0.1152 s (`cuda-1D`), 0.0744 s (`cuda-shared`) and 0.0681 s (historical `main`). The largest reduction accompanies exclusive cell ownership; the shared-memory version and final buffer-reuse version provide further gains. These comparisons measure whole versions and do not isolate individual optimizations. For `B_10`, the three non-atomic versions take 0.0757, 0.0412 and 0.0378 s; the atomic version fails all repetitions. The workloads differ: `A_1M` has one storm of 20,000 particles on one million cells, while `B_10` has ten such storms on 20,000 cells.

Figure 6 selects the lowest eligible median independently for each branch and workload. Historical `main` reaches approximately 4254× on `A_1M` at 256 threads and 1478× on `B_10` at 512 threads. The sequential baselines are medians of four cluster runs on `node149`: 289.659 and 54.342 s. These ratios compare different allocations and include all implementation differences from the sequential code; the selected blocks do not represent one fixed configuration.

3.5.2 Block-Size Choice

For historical `main`, `A_1M` medians at 256, 512 and 1024 threads are 0.0681, 0.0697 and 0.0693 s (Figure 5). For `B_10`, they are 0.0378, 0.0368 and 0.0520 s. Thus, larger blocks offer little benefit on `A_1M`, and 1024 threads substantially worsens `B_10`. The `B_10` ranges at 256 and 512 overlap, so the lowest median alone does not establish a reliable advantage.

3.5.3 Conclusions and Measurement Limits

- **Version changes matter:** the non-atomic versions reduce measured runtime, with the largest `A_1M` improvement between `cuda-first` and `cuda-1D`.
- **Block size depends on workload:** 256 threads is a reasonable common choice for these two workloads; increasing the block size does not consistently improve runtime.
- **Final-build performance remains unmeasured:** historical `main` results cannot establish the performance of the submitted grid-corrected version; a repeated campaign is still needed.

Historical `main` below 256 threads is excluded because relaxation coverage was incomplete; corrected small-block reruns concern `cuda-1D` and `cuda-shared`. Historical `main` records lack commit and executable hashes. Failed runs and timings below 0.02 s are excluded from speedup rankings. Passing requires exact peak positions and relative float tolerance 10^{-5}, a convention not explicitly specified by the exam. The measurements do not establish occupancy or bandwidth as the cause of block-size differences.

4 Profiling, Limitations and Improvements

4.1 Testing Environment

All experimental evaluations across the development branches were conducted on the HPC cluster infrastructure provided by the Department of Computer Science at Sapienza University of Rome. The cluster comprises 31 heterogeneous compute nodes divided into two primary architectural tiers:

- **CPU-Only Compute Nodes:** Powered by AMD EPYC 7301[5] processors (Naples microarchitecture), providing 32 physical cores (up to 64 logical threads depending on SMT configuration) and equipped with 252 GB of DDR4 memory, with high-memory partitions scaling up to 1004 GB.
- **Accelerated GPU Nodes:** Featuring an identical host CPU and memory subsystem, paired with one or two dedicated NVIDIA Quadro RTX 6000[6] GPUs (Turing architecture, 24 GB VRAM) utilized for the CUDA kernel benchmarks.

4.2 Measured Phase Costs

Bombardment is the longest measured phase in the selected large-workload profiles: on `A_1M`, it accounts for about 98% of wall time in both implementations (Figure 7). This is indicative evidence: the historical profiling campaign used only one run per workload/configuration, so variability is unknown and the results do not characterize the final submitted `main`.

We instrumented the phase boundaries and accumulated their durations across storms. The hybrid profiler used `MPI_Wtime`, then reported an independent `MPI_MAX` for each phase [1]; different ranks can supply these maxima, so their sum may exceed wall time. Peak reduction includes scanning, thread merging, synchronization and `MPI_Allreduce`. CUDA used paired device events, synchronizing the stop event before reading elapsed time [7]; setup and reduction windows include mixed host/API/device activity. Wall time comes from the driver. Instrumentation overhead was not measured, and these timers identify the costly phase without establishing its hardware bottleneck.

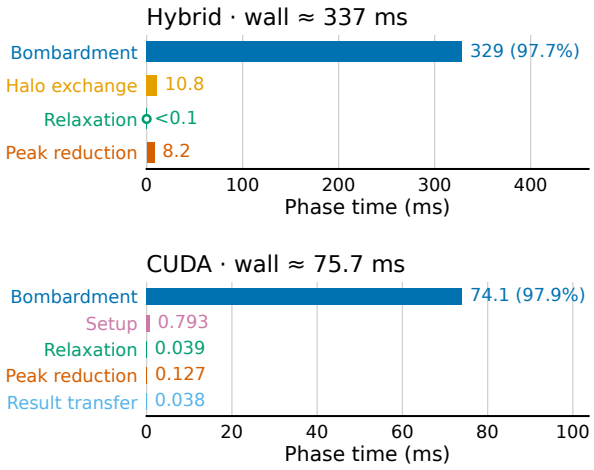


Figure 7: *Approximate `A_1M` phase times, one run/configuration (variability unknown). Hybrid: `Add_SIMD_Directives`, four nodes, eight ranks/node, four threads/rank. CUDA: `cuda-shared`, 256 threads/block, `node122`. Separate linear axes; percentages use wall time. Hybrid relaxation is below printed precision; rank maxima are not additive.*

4.3 Limitations and Targeted Next Steps

Bombardment is the first optimization priority suggested by Figure 7. Three approaches could be tested; their benefits have not yet been measured:

- **Process several cells together.** On the CPU, SIMD could calculate contributions for several independent cells at once [2, Sec. 2.8.1], preserving particle order within each cell. On the GPU, tune particle-tile and block sizes to improve reuse and execution efficiency [8]. The profiles do not establish which hardware resource limits bombardment.
- **Work while MPI messages travel.** The current code posts halo exchanges and immediately waits. Instead, compute interior cells while communication is pending, then process boundary cells once the neighboring values arrive. This could overlap communication with relaxation or peak scanning, while preserving buffer lifetimes and synchronization [1, Sec. 3.7].
- **Reduce CUDA launches and copies.** Let each thread examine more reduction candidates to reduce the number of kernel passes. Keep storm peaks on the device and copy them together at the end, replacing two blocking result copies per storm; batching small transfers is recommended in the CUDA optimization guide [8]. Reduction buffers are already reused. These changes have lower priority because the historical profiles show much smaller costs than bombardment.

Validation should use repeated runs of the final build, compare profiled and unprofiled execution, and record executable/input hashes, hardware and thread binding. Hardware counters are needed to explain resource limits. Any change must pass the sequential peak checks, including ties and threshold-sensitive inputs. Single-run historical profiles cannot quantify variability or pre-

dict gains; timings below 0.02 s remain unsuitable for speedup rankings.

5 Conclusions

The main design lesson in Sections 2.2 and 3.2 is to assign each cell to one thread. That thread processes all particle contributions for its cell, accumulates them in a local variable, and writes the result once. Since threads no longer write to the same cell, atomic operations are unnecessary. This simplifies synchronization and reduces repeated writes to the layer.

The hybrid results in Section 2.3 show that large workloads benefit more from additional workers, while communication and synchronization matter more for small inputs. The CUDA comparisons in Section 3.5 favor versions combining exclusive cell ownership, shared particle data and reusable buffers. They also show that increasing threads per block does not consistently improve performance: the choice depends on the workload.

Section 4.2 identifies bombardment as the main measured cost, making it the first optimization priority. Those profiles contain only one run per configuration and concern historical versions. Together with the coverage fix in Section 3.4, this reinforces the need to check correctness and repeat measurements on the final build before judging an optimization.

References

[1] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 3.1*. Message Passing Interface Forum. June 4, 2015. URL: https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report-book.pdf (visited on 10/10/2026).

[2] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 4.5*. OpenMP Architecture Review Board. Nov. 2015. URL: https://www.openmp.org/mp-documents/openmp-4.5.pdf (visited on 10/10/2026).

[3] Gene M. Amdahl. "Validity of the single processor approach to achieving large scale computing capabilities". In: *Proceedings of the April 18-20, 1967, spring joint computer conference*. AFIPS '67 (Spring). New York, NY, USA: ACM, 1967, pp. 483–485. DOI: 10.1145/1465482.1465560.

[4] NVIDIA Corporation. *CUDA C++ Programming Guide*. Version 12.6. NVIDIA Corporation. URL: https://docs.nvidia.com/cuda/archive/12.6.0/cuda-c-programming-guide/index.html (visited on 10/11/2026).

[5] Advanced Micro Devices, Inc. *AMD EPYC 7301 Processor Specifications*. https://www.amd.com/en/products/specifications/server-processor.html. Accessed: 2026-10-10. 2019.

[6] NVIDIA Corporation. *NVIDIA Quadro RTX 6000 Graphics Card Specifications*. https://nvidia.com. Accessed: 2026-10-10. 2018.

[7] NVIDIA Corporation. *CUDA Runtime API: Event Management*. Version 12.6. NVIDIA Corporation. URL: https://docs.nvidia.com/cuda/archive/12.6.0/cuda-runtime-api/group__CUDART__EVENT.html (visited on 10/11/2026).

[8] NVIDIA Corporation. *CUDA C++ Best Practices Guide*. Version 12.6. NVIDIA Corporation. URL: https://docs.nvidia.com/cuda/archive/12.6.0/cuda-c-best-practices-guide/index.html (visited on 10/11/2026).